A Study of Component Based Software System Metrics

Sakshi Patel, Jagdeep Kaur
Department of CSE &IT
The NorthCap University
Gurgaon 122017, India
sakshi.smk@gmail.com, jagdeep@ncuindia.edu

***Abstract*-The term metrics is basically a quantitative measure of extent to which given attributes influenced by a system, component or a process. Metrics are required to measure software quality, improve software quality and to predict software quality. There are various approaches available to develop a software metrics like object oriented approach, component based approach, distributed approach. Component based software engineering is a latest approach in developing software. The main function of component based metrics is to provide reusability and decrease cost and development time. These metrics are used for evaluating quality and managing risk. The aim of this paper is to study component based metrics. In this paper comparison of some component based metric is done on the basis of functional and non-functional characteristics of software and discussed how this new approach is different from any other approach for software development.**

Keywords: Component Based Software Engineering (CBSE), Software Metrics, Reusability, Complexity.

I. INTRODUCTION

The complexities of software application are increasing these days making traditional tools and methods insufficient for software development. As a result there is a decrease in the final quality. The most recent way for software development is Component Based Software Engineering. CBSE is process that follows standards of design and construction by using reusable software component of computer based system. It uses both Commercial-Off-The-Shelf (COTS) components and in-house components. The commercial-off the components are those components which are designed by third party. Some examples of these components are shown in figure 1. These components are tested earlier and their code cannot be modified [2, 3]. The selection of these COTS component is based on their functionality, quality, cost and degree of adjustment need to be carried out [2]. Various aspects of software complexity are determined by software metrics that's why software metrics performance vital job in improving the quality of software and managing the risk. Metrics are helpful in providing information about quality feature of software such as reusability, portability and understandability etc [4]. It also helps developer in determining possible risk so that corrective action can be taken before. The metrics play a helpful role in highlighting the system by improving its final quality. Large size software system can be developed using metrics [5]. Metrics improve reusability and provides high level of abstraction [3, 4]. This technique of using already existing components for software development has been shown in figure 2. It increases the overall quality of software system under development and also increases maintainability and productivity of resulting software. It also decrease development time, effort and of software development [1].

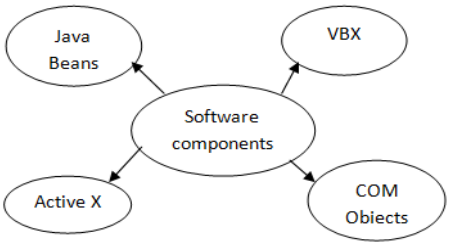

Figure 1: Software Components

There are various metrics available like product metrics but these are not sufficient for estimating complexity of components as component complexity affects time, effort, cost etc. Thus, a metrics is needed to be developed for components based system which can measure complexity of components [1, 2]. The rest of the paper is prepared as follows: Section IIdescribe component based software engineering and also about software metrics and reusability of components. Section III describes some existing research on component based software metrics. Section IV is having some component based software metrics. In section V we have done comparison of various metrics. Section VI shows conclusion of our research.

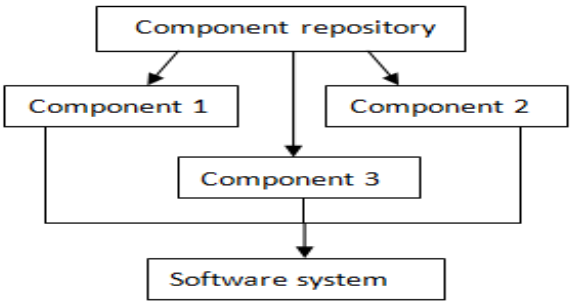

Figure 2: Component Based Techniques

ISBN:978-1-5090-1666-2/16/$31.00 ©2016 IEEE

II. COMPONENT BASED SOFTWARE ENGINEERING

Component based software engineering approach is a new approach which is widely used by much business software to estimate software quality and complexity. In component based software development approach we built systems by independently created section, known as components [4].

A. Software metrics

In this measurement based methods are applied to products, procedures and services to deliver engineering and management information to obtainsuperior products, procedures and service. Software metrics are used to measure software quality, performance etc. As we know there are various software metrics which already exists, most of them are based on source code [11]. However, these metrics cannot be used as component based metrics. So there is need of different set of metrics to estimate the quality and various other characteristics of component based system [17].

B. Software reusability

In software reusability we use existing software components to implement and update our software system. To increase the productivity, quality and reliability one must use good software reuse process. It also reduces cost and implementation time [11]. Reuse process is base of knowledge whose quality improves after each reuse and which minimize the development time and work for future projects. It helps in identifying and managing the risks earlier of new projects. Software reuse is one time investment of time and money which will recover itself in a few reuses [2].

III. LITERATURE SURVEY

There is a lot of existing research on component based software metrics. Model Joaquina Martin-Albo, Manuel F. Bertoa, Coral Calero et al.'s [2] has designed the CQM model (component Quality Model). In this they evaluate component quality on four dimensions and have main focus on usability metrics. Yu et al. give a metrics for component quality and complexity to clear difference between coupling and dependency. They suggested using separate to measure both of them. V.LakshmiNarasimhan, P.T. Parthasarathy, and M.Das [12] analyze and evaluate various metrics proposed by many researchers and conclude some useful results. MajdiAbdellatief et al.'s [5] shows the dependency between components is major factor affecting the structural design of Component Based Software System (CBSS). For this they provided dependency metrics based on components information. Prakriti Trivedi and Rajeev Kumar [4] provided a software matrix set to check the interconnection among the software components and its function. By using software component reusability, they provide a software metrics to evaluate the software quality. To make approximate calculation of software quality using software component reusability they provided a software metrics. Sachin Kumar et al.'s [1] proposed a metric which is developed using black box components. They determine the coupling complexity of software. PoojaRana and Rajender Singh [11] discovered different types of complexity metrics based on component. Component information flow metrics and component coupling metrics are the two sets of metrics which are proposed by them. A.Aloysiusi and K.Maheshwaran [15] suggested further improvement of various component based metrics.

IV. CBSE METRICS

Component based software metrics are those metrics which will measure the quality and manage the risk of component based software. To build an efficient metrics, first identify the main characteristics of work. After that divide them or break down into sub-characteristics [15]. Then these refined sub characteristics are appearing in attributes. These attributes based on metric definitions are used to get required metrics.

A. Metric Suite:

This metrics structure is represented in the form of tree. In component based software engineering there are two types of metrics: 1) Non-functional Metrics 2) Functional Metrics.

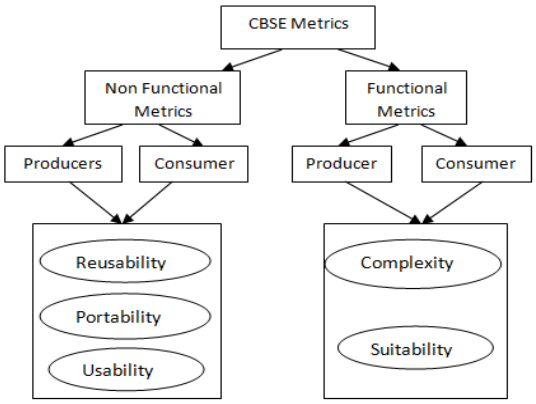

Figure 3: CBSE Metrics Tree

In this we are having various metrics based on various dimensions.

i) ***Suitability Metrics:*** The degree to which components fulfill the confine requirement. The component suitability is the nature that can be determined after the component gets installed [16]. For suitability there are two types of metrics based on different perspective:

Required Functionality (RF): It comes under producer perspective. In this only required functionality need to be checked that must be satisfied.

$$RF = \frac{\text{No. of necessary functionality given by the components}}{\text{Total no. of functionalities necessary by the component based system}}$$

Increase in the value of RF will increase suitability of component.

Extra Functionality (EF): It comes under consumer perspective. In this extra functionalities are needed to be checked.

$$EF = \frac{\text{No. of extra functionalities given by the Components}}{\text{Total no. of functionalities necessary by component based system}}$$

As the value of EF decrease the suitability will increase because increase in EF will increase the unwanted functionalities [15].

ii) ***Complexity:*** complexity of software depends upon its complexity attributes such as coupling, cohesion etc. The quality of software components, its interfaces and specifications are computed by complexity metrics. [7]. The more demand of quality will automatically increase the complexity of component. In this one metrics is based on producer perspective and two for consumer's perspective.

Component coupling (COC): It comes under producer perspective. In this internal structure of component is checked i.e. classes and relationship between them.

$$COC = \frac{\text{No. of other components sharing attribute or methods}}{\text{Total No. of possible sharing pairs in the component-based application}}$$

Interface complexity: It comes under consumer perspective. The quality characteristics such as usability, portability, performance and reusability are evaluated by complexity metrics [5]. More complex interface from user point of view will create testing and debugging problem. In this there are two metrics:

Constraints complexity (CTC):

$$CTC = \frac{\text{No. of constraints}}{\text{No. of properties and operation in an Interface}}$$

Configuration Complexity (CFC):

$$CFC = \frac{\text{No. of configuration}}{\text{No. of context of use of the components}}$$

iii) Component coupling complexity metrics for black box component:

CCCM (BB) = FICM (BB) + FOCM (BB)

Where FICM (BB) is Fan-in complexity Metric used to compute the coupling complexity due to received information from additional component and FOCM is fan-out complexity metrics which is used to compute the complexity because of leaving information [15, 16].

iv) Reusability: The degree to which a component can be used reused by software and some given application. It is the quality of software to improve productivity [5]. There are various sub factors of reusability as shown in figure 4.

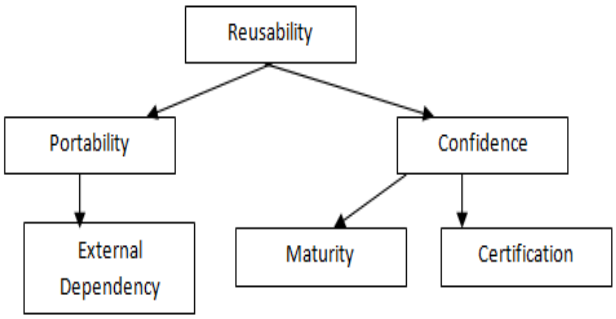

Figure 4: Component Reusability Tree

Portability: In this external dependency is evaluated.

$$ED = \frac{\text{No. of parameter and methods passed and return values}}{\text{Total No. of methods (Read/ Write)}}$$

Confidence: In this maturity level of reusable component is calculated

Mat= DF + CR

DF= No. of faults detected

CR= No. of changes Requests [15].

V. QUALITY EVALUATION OF COMPONENT BASED SOFTWARE SYSTEM

There are many metrics proposed by many researchers to assess CBSS attributes. These metrics can be hard to use in real. Like many of the metrics are not well described or not have clear concept, which could hamper their execution. In this section existing study ofmetrics for component based is understand, classify and analyzed to assess the quality of CBSS from components customer's point of view.

A. Research Method:

i) Protocol Development: In this first protocol is designed for methodical mapping that give attention toquestions that are associated to the assessment of CBSS quality.

ii) Research queries and motivation**:** There are three research queries which have been addressed here:

RQ1: Are the measurement applied on complete CBSS or on individual components?

In this basically granularity level of the metrics is defined.

RQ2: Which component of CBSS is being measured? How they are validated?

RQ3: Are there any restrictions on present research?

iii) Search procedure and inclusion and exclusion criteria:

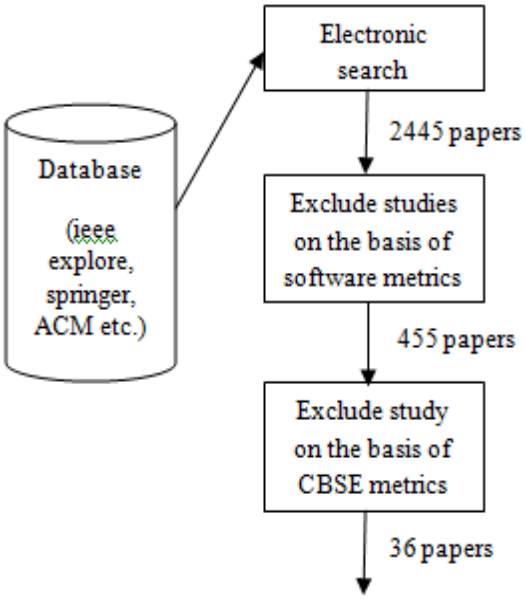

Figure 5: Search procedure and inclusion and exclusion criteria

iv) Quality evaluation questions of primary studies: In this section there are different QAQs i.e. quality assessment questions on their basis quality of CBSS is evaluated. QAQs are basically used to present a brief summary of quality concepts, objective and analysis.

v) Data Mining:In this section, all the information that is related to question defined in previous step is collected. This information will include:

1. Whether it is a single component or complete CBSS on which test is performed.
2. Complete name of metrics with acronyms.
3. Check the level at which metrics is collected i.e. component level or CBSS level.
4. What are the component models for which metrics are being proposed?
5. Metrics specifications.
6. Metrics assumptions and understanding guidelines.

vi) Data Examination:In this first granularity level of metrics is examined. After that classification is extended with respect to application of individual metrics to single component or complete CBSS based metric calculation and description. Data is mined in previous section with respect to RQs and QAs.

VI. COMPARISON OF VARIOUS METRICS

TABLE 1: Metrics for various component

components / Metrics	Interface Complexity Metrics	Quality Metric using reusability	Dependency
COC	--	Yes	--
CTC	Yes	--	--
CFC	Yes	--	--
Reusability	--	--	--
Suitability	--	--	--
Dependency	--	--	Yes
Usability	--	Yes	--
CCCM (BB)	Yes	--	--

TABLE 2: Metrics for various component

components / Metrics	Black box component complexity metrics	A metrics group for evaluating software components
COC	--	--
CTC	--	--
CFC	--	--
Reusability	--	Yes
Suitability	--	Yes
Dependency	--	--
Usability	--	--
CCCM (BB)	Yes	--

This table 1and table 2 describes the various component based metrics available on different parameters. Yes signifies that which metric can measure which characteristics of software component. In this, there are some metric which can measure more than one characteristics of software component also. In this, for particular characteristics many metrics are available, according to requirements metrics can be selected among them.

VII. CONCLUSION

This paper presents an overview on component based software metrics. In this there are many metrics for one characteristics of component based software. For checking complexity of component at interfaces there are three metrics. CCCM (BB) is the best metrics. CCCM is black box testing metrics. As it is interface complexity metrics it estimates testing, understandability and effort. This paper discussed some of important reusability metrics which cover both functional and non-functional aspects of software system. These metrics help in estimating quality characteristics of software. This paper discussed and reviewed the metrics based on component. According to this research, there are very few reusability metrics which are clearly defined and validated. To use them in future these metrics need to be validated first.

REFERENCES

[1] Sachin Kumar, Pradeep Tomar, Reetika Nagar, SuchitaYadav,"Coupling Metric to Measure the Complexity of Component Based Software through Interfaces", April 2014.

[2] Joaquina Martín-Albo, Manuel F. Bertoa, Coral Calero, Antonio Vallecillo, Alejandra Cechich and Mario Piattini "CQM: A Software Component Metric Classification Model" IEEE TRANSACTIONS JOURNAL, 2003.

[3] ChanderDiwaker, Sonam Rani, Pradeep Tomar "Metrics Used In Component Based Software Engineering", ICFTEM-2014.

[4] Prakriti Trivedi, Rajeev Kumar, "Software Metrics to Estimate Software Quality using Software Component Reusability", IJCSI International Journal of Computer Science Issues, Vol. 9, Issue 2, No 2, March 2012.

[5] MajdiAbdellatief∗, Abu BakarMd Sultan, Abdul Azim Abdul Ghani1, Marzanah A. Jabar, "A mapping study to investigate component-based software system metrics", jo u rn al homepage: www.elsevier.com, 5 October 2012.

[6] TullioVernazza, GiampieroGranatella, Giancarlo Succi, Luigi Benedicenti, Martin Mintchev, "Defining metrics for software components", July 2003.

[7] MajdiAbdellatiefab, Abu BakarMd Sultana, Abdul AzimAbdGhania, MarzanahA.Jabara, "Component-based Software System Dependency Metrics based on Component Information Flow Measurements" ICSEA: The Sixth International Conference on Software Engineering Advances, 2012.

[8] DanailHristov, Oliver Hummel, MahmudulHuq, Werner Janjic, "Structuring Software Reusability Metrics for Component-Based Software Development", ICSEA: The Seventh International Conference on Software Engineering Advances, 2012.

[9] N. Gehlot and J.Kaur, "Dynamic Inheritance Coupling Metric-Design and Analysis for Assessing Reusability", Int. J. Software Engineering Technology and Applications, Vol.1, No.1, PP. 118-133, 2015

[10] V. Subedha, S. Sridhar, "Design of Dynamic Component Reuse and Reusability Metrics Library for Reusable Software Components in Context Level", February 2012.

[11] PoojaRanaRajender Singh, "A Study of Component Based Complexity Metrics", November 2014.

[12] V. L. Narasimhan and B. Hendradjaya, "A New Suite of Metrics for the Integration of Software Components", 2007

[13] P. Edith Linda, V. ManjuBashini, S. Gomathi, "Metrics for Component Based Measurement Tools", International Journal of Scientific & Engineering Research Volume 2, Issue 5, May-2011.

[14] Vinay Tiwari, Dr. R.K. Pandey, "Open Source Software and Reliability Metrics", International Journal of Advanced Research in Computer and Communication Engineering Vol. 1, Issue 10, December 2012.

[15] A.Aloysius and K.Maheswaran, "A Review on Component Based Software Metrics", 22 January 2015

[16] V. Lakshmi Narasimhan, P. T. Parthasarathy, and M. Das, "Evaluation of a Suite of Metrics for Component Based Software Engineering (CBSE)", 2009

[17] Hironori Washizaki, HirokazuYamamotoandYoshiakiFukazawa,"A Metrics Suite for Measuring Reusability of Software Components", 2005.

[18] K.P. Srinivasan1 and T. Devi, "Software Metrics Validation Methodologies InSoftwareEngineering", International Journal of Software Engineering & Applications (IJSEA), Vol.5, No.6, November 2014.